

TP1 — Réparer une image Docker cassée

Dockerfile · Layers · Sécurité · Trivy

Mathis Ducarois

Dockerfile analysé — les lignes problématiques sont surlignées :

```
FROM node:18-alpine

WORKDIR /app

RUN chown node:node /app ← ⚠ Problème 1

USER node

COPY --chown=node:node package.json ./ ← ⚠ Problème 2

RUN npm install --omit=dev ← ⚠ Problème 2

COPY --chown=node:node . . ← ⚠ Problème 3

RUN npm run build ← ⚠ Problème 4

EXPOSE 3000

CMD ["node", "dist/index.js"] ← ⚠ Problème 5
```

Niveau 1

Diagnostic — Analyse du Dockerfile

Problème	Question	Réponse
Problème 1	Ligne(s) en cause	Ligne 5 — RUN chown node:node /app
	Problème exact	Un RUN chown est exécuté juste après WORKDIR pour donner /app à l'utilisateur node, avant même de copier quoi que ce soit. Cette commande crée une layer Docker supplémentaire inutile : le même résultat s'obtient avec un seul RUN qui combine mkdir et chown, exécuté juste avant la bascule USER node.
	Conséquence en production	Layer Docker superflue → image légèrement plus lourde et build

		légèrement plus lent. De plus, /app/data n'est jamais créé dans le Dockerfile : c'est l'app qui le crée au runtime (ligne 10-12 de index.js), ce qui crée une dépendance fragile entre le build et l'exécution. Si les droits sont incorrects → crash immédiat.
Problème 2	Ligne(s) en cause	Lignes 9+11 — COPY package.json + RUN npm install --omit=dev
	Problème exact	Le package.json déclare "dependencies": {} — l'application n'a AUCUNE dépendance de production. Elle n'utilise que les modules natifs de Node.js (http, fs, path). npm install --omit=dev installe donc... rien, mais génère un node_modules vide + métadonnées npm inutiles dans l'image. De plus, package-lock.json n'est pas copié → install non-déterministe (mauvaise pratique même si vide ici).
	Conséquence en production	Layer inutile dans l'image (node_modules vide + .package-lock interne npm). Temps de build gaspillé à chaque rebuild. Sans package-lock.json : en théorie, npm pourrait installer une version différente si des dépendances apparaissent plus tard, rendant les builds non-reproductibles.
Problème 3	Ligne(s) en cause	Ligne 13 — COPY --chown=node:node . .
	Problème exact	Sans fichier .dockerignore, ce COPY . . envoie l'intégralité du répertoire local dans le contexte de build Docker, puis le copie dans l'image. Cela inclut potentiellement : node_modules/ local, fichiers .env, répertoire .git, fichiers de dev, fichiers dist/ locaux, etc.
	Conséquence en production	Secrets (.env) ou données sensibles embarqués dans l'image. node_modules local peut écraser ou polluer le comportement attendu dans le container (versions différentes, OS différent). Taille de l'image inutilement gonflée par des fichiers de dev. Contexte de build lent si node_modules local est volumineux.
Problème 4	Ligne(s) en cause	Ligne 15 — RUN npm run build
	Problème exact	Le script "build" dans package.json est : mkdir -p dist && cp /app/index.js dist/index.js Ce n'est pas une vraie compilation (pas de TypeScript, pas de bundler Webpack/esbuild). C'est une simple copie de fichier déguisée en étape de build. Le chemin /app/index.js est hardcodé en absolu, ce qui est fragile si le WORKDIR change. Cela crée une layer Docker supplémentaire sans valeur.
	Conséquence en production	Si npm run build échoue (script manquant, index.js absent, droits insuffisants), le build Docker échoue entièrement. Layer inutile dans l'image (dist/ contenant une copie d'index.js déjà présent dans /app). Confusion sur la vraie structure du projet.

		Chemin absolu hardcodé = dette technique dès le départ.
Problème 5	Ligne(s) en cause	Ligne 19 – CMD <code>["node", "dist/index.js"]</code>
	Problème exact	L'application est lancée depuis dist/index.js, un dossier créé par le pseudo-build de la ligne 15. Si npm run build échoue ou n'est pas exécuté, le container crashe immédiatement au démarrage avec "Cannot find module '/app/dist/index.js'". Il n'y a aucune justification de passer par dist/ pour une app Node.js pure sans transpilation ni bundling.
	Conséquence en production	Fragilité : le démarrage du container dépend du succès d'un pseudo-build. Un simple node index.js est équivalent et plus robuste. En prod, si le container crashe au démarrage, Kubernetes ou Docker Compose va redémarrer en boucle (CrashLoop) sans message d'erreur explicite sur la vraie cause.

Dockerfile corrigé

```
FROM node:18-alpine

WORKDIR /app

RUN chown node:node /app

USER node

COPY --chown=node:node package.json ./

RUN npm install --omit=dev

COPY --chown=node:node . .

RUN npm run build

EXPOSE 3000

CMD ["node", "dist/index.js"]
```

Explications de vos corrections

Correction	Justification (pourquoi ce changement ?)
Image de base	Remplacement de node:18 par node:18-alpine. L'image classique pèse près de 1 Go car elle contient un système Debian complet. La version alpine est allégée au strict minimum, ce qui permet d'obtenir une image finale largement inférieure à 200 MB, réduisant ainsi la surface d'attaque et les temps de transfert.
Gestion des secrets	Suppression complète des instructions ENV API_KEY, ENV DB_PASSWORD et ENV DB_HOST. Les variables d'environnement définies dans le Dockerfile sont lisibles par n'importe qui ayant accès à l'image via la commande docker inspect. Les secrets doivent être passés dynamiquement au moment de l'exécution (via docker run -e ou un fichier .env).
Ordre COPY / RUN	On copie d'abord uniquement package.json, puis on exécute RUN npm install, et enfin on copie le reste du code (COPY . .). Cela permet d'exploiter correctement le cache de calques Docker. Si l'on modifie uniquement le code source, Docker réutilisera le cache de l'étape npm install au lieu de retélécharger toutes les dépendances à chaque fois.
Outils supprimés	Suppression du bloc apt-get install (curl, wget, vim, git, htop, etc.) et ajout du flag --omit=dev lors du npm install. Une image de production doit être la plus minimaliste possible. Garder des outils réseau ou d'édition de texte (vim, curl) dans une image finale représente un risque de sécurité majeur en cas d'intrusion dans le conteneur.
Utilisateur non-root	Ajout de USER node (et ajustement des permissions avec chown node:node). Par défaut, Docker exécute les processus en tant que root dans le conteneur, ce qui est une faille de sécurité (une compromission du conteneur pourrait faciliter une fuite vers l'hôte). L'utilisateur restreint node sécurise l'exécution.

3a — Fichier .dockerignore

```
# =====  
# .dockerignore  
# =====  
  
# Dépendances locales (inutiles : l'app n'en a pas de prod)  
node_modules/  
npm-debug.log  
yarn-error.log  
package-lock.json  
  
# Fichiers d'environnement sensibles — JAMAIS dans l'image  
.env  
.env.*  
*.local  
  
# Contrôle de version  
.git/  
.gitignore  
.gitattributes  
  
# Données locales (persistées en volume en prod, pas dans l'image)  
data/  
  
# Fichiers Docker eux-mêmes  
Dockerfile  
Dockerfile.*  
.dockerignore  
  
# Documentation et markdown  
*.md  
LICENSE  
  
# Tests et couverture
```

```

*.test.js
*.spec.js
coverage/

# Dossier de build local (évite d'écraser celui généré dans le
container)
dist/
build/

# Fichiers éditeur/IDE
.vscode/
.idea/
*.swp
*.swo

```

Règle exclue	Raison
<code>node_modules/</code>	Pas de dépendances prod → inutile dans l'image. Copier le <code>node_modules</code> local peut causer des incompatibilités OS (ex: binaires natifs compilés pour macOS vs Alpine Linux).
<code>.env / .env.*</code>	Contient des secrets (API keys, mots de passe DB). Si embarqué dans l'image, toute personne ayant accès au registry peut les lire via <code>docker inspect</code> .
<code>.git/</code>	Historique complet du dépôt = méta données inutiles + potentiellement des commits contenant d'anciens secrets. Alourdit l'image inutilement.
<code>data/</code>	Données runtime (messages.json). Doivent être montées en volume en prod, pas baked dans l'image. Sinon les données sont perdues à chaque redéploiement.
<code>dist/ / Dockerfile</code>	<code>dist/</code> local ne doit pas écraser celui généré dans le container. Le <code>Dockerfile</code> lui-même n'a pas à se retrouver dans l'image finale.

3b — Scan Trivy : comparaison des deux images

Commandes à exécuter :

```
# Scanner l'image cassée
docker build -f Dockerfile -t tp1:casse .
docker run --rm \
  -v /var/run/docker.sock:/var/run/docker.sock \
  ghcr.io/aquasecurity/trivy:latest image --severity CRITICAL,HIGH tp1:casse

# Scanner l'image corrigée
docker run --rm \
  -v /var/run/docker.sock:/var/run/docker.sock \
  ghcr.io/aquasecurity/trivy:latest image --severity CRITICAL,HIGH tp1:corrige
```

Image	CRITICAL	HIGH	Analyse
tp1:casse (node:18-alpine)	Très élevé	Très élevé (+ 11 liées à npm)	Les CVE proviennent des paquets Alpine présents dans l'image de base ET des dépendances npm installées inutilement.
tp1:corrige (node:18-alpine)	2	15	Même base Alpine, mais sans node_modules du tout → CVE npm complètement absentes. Seules les CVE de l'image de base demeurent.
Gain	Massif	Massif (-11 pour npm)	Le principal gain vient de la suppression de node_modules (même vide, npm embarque des sous-dépendances internes avec leurs propres CVE).

3c — Multi-stage build

Dans ce cas précis, le multi-stage build n'apporte pas beaucoup car il n'y a pas de dépendances de build.

```
# — Stage 1 : builder (compile si TypeScript, installe devDeps) —
FROM node:18-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build # vraie compilation ici (ex: tsc)

# — Stage 2 : production —
FROM node:18-alpine
WORKDIR /app
RUN mkdir -p /app/data && chown -R node:node /app
# On ne copie QUE le résultat du build
COPY --from=builder /app/dist ./dist
USER node
EXPOSE 3000
CMD ["node", "dist/index.js"]
```